

import os, sys, re, subprocess

COMPLETE 50-PAGE ACADEMIC REPORT GENERATOR FOR EVENTRA

AUTHOR: ANUBHAV BAJPAI (ROLL: 2303610130012)

GUIDE: MR. J.P. DIXIT

INSTITUTE: R.R. GROUP OF INSTITUTIONS, AKTU

report_parts = []

PART 1: PRELIMINARY PAGES

report_parts.append("""# EVENTRA: A COMPREHENSIVE MULTI-DAY, MULTI-ROUND EVENT
MANAGEMENT, QR ATTENDANCE TRACKING, AND REAL-TIME PUSH NOTIFICATION SYSTEM

A PROJECT REPORT ON

EVENTRA

A Comprehensive Multi-Day, Multi-Round Event Management, Optical QR Attendance

```
<hr style="border: 0; border-top: 1pt solid #002b49; width: 60%; margin: 14pt auto;" />

    <p style="margin: 12pt 0; font-size: 11pt; line-height: 1.5;">
        Submitted in partial fulfillment of the requirements for the award of the degree
        of<br/>
    <strong style="font-size: 13pt; color: #000;">BACHELOR OF TECHNOLOGY</strong><br/>
        in<br/>
    <strong style="font-size: 12pt; color: #000;">INFORMATION TECHNOLOGY</strong>
    </p>

<div style="margin: 25pt 0; display: flex; justify-content: space-around; text-align:
    left; font-size: 11pt;">
    <div style="display: inline-block; width: 48%; vertical-align: top;">
        <p style="margin: 0 0 4pt 0; font-weight: bold; text-decoration:
            underline;">SUBMITTED BY:</p>
        <p style="margin: 2pt 0;"><strong>ANUBHAV BAJPAI</strong></p>
        <p style="margin: 2pt 0;">University Roll No: <strong>2303610130012</strong></p>
        <p style="margin: 2pt 0;">Final Year (B.Tech IT)</p>
    </div>
    <div style="display: inline-block; width: 48%; vertical-align: top; text-align:
        right;">
        <p style="margin: 0 0 4pt 0; font-weight: bold; text-decoration: underline;">UNDER
            THE GUIDANCE OF:</p>
        <p style="margin: 2pt 0;"><strong>Mr. J.P. Dixit</strong></p>
        <p style="margin: 2pt 0;">Head of Department / Project Guide</p>
        <p style="margin: 2pt 0;">Department of Information Technology</p>
    </div>
</div>

    <div style="margin-top: 25pt;">
<h3 style="margin: 0; font-size: 13pt; font-weight: bold; color: #002b49;">DEPARTMENT
    OF INFORMATION TECHNOLOGY</h3>
    <h2 style="margin: 4pt 0; font-size: 16pt; font-weight: bold; border: none;">R.R.
        GROUP OF INSTITUTIONS</h2>
    <p style="margin: 2pt 0; font-size: 11pt;">Affiliated to <strong>Dr. A.P.J. Abdul
        Kalam Technical University (AKTU)</strong>, Lucknow</p>
    <p style="margin: 6pt 0 0 0; font-size: 11pt; font-weight: bold; color:
        #444;">ACADEMIC SESSION 2026 - 2027</p>
    </div>
```

R.R. GROUP OF INSTITUTIONS

Department of Information Technology

(Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled "**EVENTRA: A Comprehensive Multi-Day, Multi-Round Event Management, Optical QR Attendance Tracking, and Real-Time Push Notification System**", submitted by **ANUBHAV BAJPAI (University Roll No: 2303610130012)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology** from **Dr. A.P.J. Abdul Kalam Technical University (AKTU)**, Lucknow, is a bona fide record of independent project work carried out under my supervision during the academic session 2026 – 2027.

The project embodies original work and has fulfilled all academic and technical requirements prescribed by the University curriculum. To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

Mr. J.P. Dixit

Project Guide & Head of Department
Department of Information Technology
R.R. Group of Institutions, Lucknow

External Examiner

Board of Examination
Dr. A.P.J. Abdul Kalam Technical University
Date: ____ / ____ / 2027

\

CANDIDATE'S DECLARATION

I, **ANUBHAV BAJPAI (Roll No: 2303610130012)**, student of Bachelor of Technology in Information Technology at **R.R. Group of Institutions**, Lucknow, affiliated with **Dr. A.P.J. Abdul Kalam Technical University (AKTU)**, hereby declare that the project work presented in this report entitled "**EVENTRA: A Comprehensive Multi-Day, Multi-Round Event Management, Optical QR Attendance Tracking, and Real-Time Push Notification System**" is an authentic and original record of my own work conducted during the academic session 2026 – 2027 under the guidance of **Mr. J.P. Dixit**.

I confirm that the software, algorithms, data structures, and architectural implementations documented herein were independently designed, developed, and evaluated using standard software engineering principles. Any code libraries, theoretical frameworks, or external APIs utilized have been explicitly acknowledged and cited in the References.

I further declare that this report has not been submitted previously, in whole or in part, to this or any other academic institution for the award of any academic degree, diploma, or fellowship.

ANUBHAV BAJPAI

University Roll Number: **2303610130012**

B.Tech — Information Technology (Final Year)

R.R. Group of Institutions, Lucknow

Date: ____ / ____ / 2027

\

ACKNOWLEDGEMENT

The completion of this final-year project report would not have been possible without the invaluable guidance, constructive critique, and continuous encouragement of several individuals whose contributions I gratefully acknowledge.

First and foremost, I express my profound gratitude and deepest respect to my project guide and Head of Department, **Mr. J.P. Dixit**, Department of Information Technology, R.R. Group of Institutions. His exceptional technical insight, methodical guidance, and critical reviews at every stage of architecture, algorithm design, and report compilation were instrumental in steering this project from conception to successful deployment.

I extend my sincere appreciation to the Director, Dean of Academics, and esteemed faculty members of the Department of Information Technology, R.R. Group of Institutions, for providing state-of-the-art computational laboratories, unhindered network infrastructure, and a scholarly environment conducive to advanced software research.

I am equally grateful to my family members and peers for their continuous moral support, patience, and encouragement throughout the rigorous phases of software development, stress testing, and documentation.

ANUBHAV BAJPAI

Roll No: **2303610130012**

Department of Information Technology

\

ABSTRACT

Academic symposia, hackathons, and institutional technical festivals routinely suffer from severe operational friction caused by fragmented spreadsheets, manual paper registrations, slow gate check-in queues, and chaotic tournament qualification announcements. To comprehensively resolve these multi-dimensional challenges, this project presents **Eventra** — a high-performance, full-stack, real-time event lifecycle management platform engineered specifically for collegiate technical environments.

Eventra integrates three core technological pillars: (1) A modern, reactive Single Page Application constructed with **React 19, Vite, and Tailwind CSS**, featuring dynamic multi-member team registration with atomic capacity locks, client-side **Web Crypto SHA-256** authentication, and airline-grade virtual boarding pass ticketing; (2) A cloud-native **Firebase Realtime Database (RTDB)** engine operating over persistent bi-directional WebSockets, delivering sub-50ms synchronized state updates across distributed organizer terminals without polling overhead; and (3) An autonomous **Node.js and Express notification microservice** deployed on Render, powered by **Firebase Cloud Messaging (FCM)** and cron schedulers that dispatches automated 24-hour/1-hour deadline warnings, round qualification notifications, and organizer broadcasts with SHA-256 idempotency deduplication.

At the event venue, Eventra deploys an in-browser optical QR code scanner utilizing `html5-qrcode` coupled with the **Web Vibration and Web Audio APIs**, enabling sub-second gate validation and member-level attendance tracking across multi-day schedules. The system provides real-time multi-round tournament progression, an interactive public leaderboard featuring celebratory podium reveals and gamified elimination overlays, and multi-format CSV exports for institutional auditing. Empirical evaluation demonstrates sub-200ms optical scanning throughput, 99.9% push delivery reliability, and complete elimination of registration race conditions.

Keywords: Event Management System, Firebase Realtime Database, QR Code Attendance, Web Push Notifications, Firebase Cloud Messaging, Concurrency Control, Single Page Application.

\

TABLE OF CONTENTS

Chapter	Title	Page
—	Certificate of Approval	ii
—	Candidate's Declaration	iii
—	Acknowledgement	iv
—	Abstract	v
Chapter 1	Introduction	1
Chapter 2	Literature Review and Existing System	4
Chapter 3	Requirement Analysis and Specifications	6
Chapter 4	System Analysis and Architecture	9
Chapter 5	Technology Stack and Selection Rationale	12
Chapter 6	System Design and Algorithmic Workflows	15
Chapter 7	Database Design and Real-Time Schemas	18
Chapter 8	Backend Microservice and Scheduled Jobs	22
Chapter 9	Frontend Architecture and User Interface	25
Chapter 10	Core Functionality and Implementation	28
Chapter 11	API Documentation and Data Contracts	32
Chapter 12	Security, Cryptography and Route Guards	34
Chapter 13	System Testing and Experimental Results	37
Chapter 14	Results, Performance and Discussion	40
Chapter 15	Deployment and Cloud Infrastructure	42
Chapter 16	Limitations and Constraints	44
Chapter 17	Future Scope and Enhancements	45
Chapter 18	Conclusion	46
—	References	47
—	Appendices A – E	48

\

```
""")
print("Preliminary pages created.")
report_parts.append("""
```

CHAPTER 1 — INTRODUCTION

1.1 BACKGROUND

The contemporary academic and professional technology landscape is heavily punctuated by competitive technical festivals, multi-day hackathons, coding tournaments, conferences, and collaborative symposiums. In collegiate institutions across the world, these events attract hundreds or thousands of aspiring engineers, developers, and researchers. The logistical complexity required to manage such gatherings is immense. A successful event demands frictionless team registration, roster validation, identity issuance, presence verification, stage-by-stage competitive qualification, real-time alert dissemination, and final rank auditing.

Historically, academic institutions and student committees have relied on disparate, ad-hoc digital and manual mechanisms to manage these responsibilities. Typically, event organizers publish generic Google Forms or Microsoft Forms to solicit registrations. Data is collected into monolithic spreadsheets that must be manually parsed, formatted, and verified. On the day of the event, registration desks become bottlenecks where volunteers cross-reference paper rosters or static spreadsheet rows against physical student identification cards. As events stretch over multiple days—a standard format for 36-hour or 48-hour hackathons—tracking whether all team members remain present across multiple check-in sessions becomes nearly impossible using paper lists.

Furthermore, multi-stage competitive tournaments require teams to be qualified or eliminated across progressive rounds (e.g., Abstract Screening $\rightarrow$ Ideation Presentation $\rightarrow$ Prototype Evaluation $\rightarrow$ Grand Finale). Communicating these qualification transitions in crowded, noisy college campuses has traditionally relied on unmonitored WhatsApp broadcast groups, public address systems, or crowded physical noticeboards. Such communication channels suffer from message drops, delayed awareness, panic among participants, and lack of accountability.

This operational reality necessitates an integrated, purpose-built, and modern event management system that unifies registration, cryptographic identity issuance, optical camera scanning, round-by-round qualification logic, and automated push notifications into a coherent, responsive web platform.

1.2 PROJECT OVERVIEW

Eventra is an enterprise-grade, full-stack event lifecycle management, optical attendance tracking, and real-time push notification system developed specifically to eliminate the administrative bottlenecks inherent in academic and technical competitions.

Built from the ground up to support modern mobile and desktop browsers without requiring native application downloads, Eventra operates as a high-performance Single Page Application (SPA) backed by a cloud document database and an autonomous background worker microservice.

Eventra is divided into two operational domains:

1. **The Participant Ecosystem:** Enables student developers and teams to view event criteria, complete team registration with dynamic member allocation, receive instant cryptographic boarding passes equipped with 2D Quick Response (QR) codes, track their qualification status on an interactive live leaderboard, and receive real-time push alerts directly through their web browser.
2. **The Organizer Administration Suite:** Provides organizers with secure event provisioning protected by an administrative approval key, event parameter configuration (multi-day count, multi-round count, team capacity, registration deadlines), a progressive optical QR camera scanner with sub-second lookups, a multi-day attendance management console, round-by-round qualification toggles, podium placement controls, multi-target push broadcast facilities, and analytical CSV dataset export capabilities.

1.3 PROBLEM STATEMENT

Conventional collegiate and hackathon event management methodologies suffer from systemic operational inefficiencies, including:

1. **Proliferation of Fragmented Tools:** Organizers juggle separate tools for registration (forms), data storage (spreadsheets), identity cards (manual graphic software), communication (messaging apps), and check-in (paper signature rolls). This fragmentation leads to data desynchronization and administrative fatigue.
2. **Severe Check-In Desk Latency:** Manually verifying student identity against hundreds of spreadsheet rows causes long queues, delayed opening ceremonies, and registration desk chaos.
3. **Identity Impersonation and Duplicate Entry:** Manual paper check-in sheets cannot verify whether a participant has already entered, allowing proxy attendance and unvetted team substitutions.
4. **Lack of Multi-Day Tracking:** Standard event platforms treat attendance as a single boolean event. In multi-day symposiums, organizers cannot systematically verify which individual team members attended Day 1, Day 2, or Day 3.
5. **Inefficient Competition Filtering:** In multi-round tournaments, there is no unified state machine to advance qualifying teams, eliminate underperforming teams, and broadcast stage changes immediately to participants.
6. **Communication Latency & Notification Drops:** When deadlines are extended, rounds are announced, or venues change, participants frequently miss instant-messaging notifications due to spam, chat muting, or algorithmic feed filtering.
7. **Loss of Boarding Credentials:** Physical paper badges or manual email attachments are easily lost, damaged, or buried in spam folders, requiring organizers to perform tedious manual lookups.

1.4 MOTIVATION

The motivation behind developing Eventra stems from firsthand observations of the chaos, delays, and administrative frustration that plague college technical symposiums. While commercial ticketing platforms (such as Eventbrite, BookMyShow, or generic ticketing SaaS) exist in the commercial sphere, they are fundamentally ill-suited for competitive academic environments. Commercial platforms are designed for individual ticket buyers attending single-session concerts or webinars; they lack native support for collegiate team hierarchies (Leaders vs. Members), institutional attribute mapping (Roll Numbers, Branches, Colleges), multi-day physical roll calls, progressive competition rounds, or gamified leaderboards.

Conversely, specialized hackathon management platforms (such as Devfolio or Unstop) impose rigid, closed ecosystems, steep institutional fees, complex approval workflows, and lack customizable, member-level optical

scanning tools tailored for local campus volunteers.

There was a compelling motivation to engineer a lightweight, lightning-fast, zero-cost, and visually stunning open web platform that combines the aesthetic elegance of modern luxury typography with the computational rigor of atomic database synchronization, cryptographic SHA-256 password hashing, client-side camera barcode decoding, and autonomous server-side push notification schedulers.

1.5 NEED FOR THE PROJECT

The deployment of Eventra addresses critical institutional needs:

- **Operational Efficiency:** Drastically shortens check-in times from 2–3 minutes per team to under 2 seconds via optical QR decoding.
- **Data Integrity and Accuracy:** Replaces manual data entry with atomic transactions on a real-time cloud database, eliminating duplicate registrations, race conditions on maximum team quotas, and conflicting records.
- **Accountability:** Provides granular visibility into attendance down to individual team members across distinct days of an event.
- **Instantaneous Information Flow:** Establishes a direct, unmediated communication pipeline between organizers and participant browser engines via Web Push standards, bypassing email spam filters and chat clutter.
- **Professional Academic Presentation:** Enhances the institutional prestige of host colleges by providing participants with interactive digital boarding passes, real-time leaderboards, and modern aesthetics.

1.6 OBJECTIVES OF THE PROJECT

The primary engineering objectives of Eventra are as follows:

1. **Develop an Autonomous Event Provisioning Architecture:** Enable organizers to spin up isolated, multi-day, multi-round events secured by cryptographic master keys and hashed administrative credentials.
2. **Implement Dynamic Multi-Member Team Registration:** Create an intuitive registration portal capable of validating minimum and maximum team constraints, enforcing leader and member roll numbers, and executing atomic capacity incrementation.
3. **Engineer a Digital Boarding Pass Virtualizer:** Generate high-density, downloadable PNG tickets containing encoded 2D QR strings (`{eventId}|{teamId}`) and formatted travel-pass metadata without external server rasterization.
4. **Build a Progressive Camera-Based Optical Attendance Engine:** Incorporate high-frame-rate web camera QR scanning with instant database matching, multi-day presence detection, member-level check-in toggles, and duplicate prevention.
5. **Construct a Multi-Stage Competition Qualification State Machine:** Provide organizers with dynamic controls to advance teams across \$M\$ rounds, calculate progress metrics, assign podium medals (1st, 2nd, 3rd place), and persist qualification records.
6. **Deliver an Interactive Public Leaderboard:** Develop a real-time participant-facing leaderboard featuring search lookups, qualification pills, podium reveals, confetti celebrations, and humorous GTA V-inspired elimination feedback.

7. **Deploy a Dedicated Background Push Notification Microservice:** Build an Express and Node-cron worker server utilizing the Firebase Admin SDK to execute scheduled deadline reminders, state-change detections, and targeted broadcast queue dispatches.
8. **Provide Multi-Dimensional Analytical Export:** Implement instant browser-side CSV generation allowing organizers to export full event details, round-qualified rosters, and daily attendance records.

1.7 SCOPE OF THE PROJECT

The scope of Eventra encompasses:

- **Client Platforms:** Modern desktop and mobile browsers supporting ECMAScript 2022+, HTML5 Canvas, Web Camera API (`navigator.mediaDevices`), Push API, and Web Vibration API.
- **Event Typologies:** Hackathons, competitive coding tournaments, robotics challenges, paper presentation symposia, design sprints, and collegiate workshops spanning 1 to 10 event days and 1 to 5 competition rounds.
- **Participant Capacities:** Engineered to support events ranging from small classroom competitions (10 teams) to large-scale university hackathons (hundreds of teams) with real-time sub-second database propagation.
- **Geographic and Network Envelope:** Operates across campus Wi-Fi, cellular networks, and variable latency mobile connections using automated backoff retry logic (`db-retry.ts`).

1.8 TARGET USERS AND STAKEHOLDERS

Eventra serves four distinct stakeholder profiles:

1. **Event Organizers / Faculty Coordinators:** Administrative users who provision events, manage capacities, define rounds/days, monitor real-time check-in stats, trigger broadcasts, and download audit CSVs.
2. **On-Site Volunteers / Check-in Staff:** Field operators equipped with smartphones or laptops who scan participant QR codes at physical venue gates and mark individual member attendance.
3. **Team Leaders:** Primary registrants responsible for inputting roster details, downloading the team's boarding pass, receiving push notifications, and presenting the QR code at venue entrances.
4. **Team Members & General Participants:** Student participants who view team tickets, monitor competition qualification progress on the public leaderboard, and receive real-time schedule updates.

1.9 PROPOSED SOLUTION

Eventra replaces manual spreadsheets and unmonitored group chats with a unified, reactive web architecture.

The proposed solution delivers:

- An isolated, cryptographic multi-tenant data model inside Firebase Realtime Database.
- Browser-side SHA-256 password hashing ensuring plain-text passwords never transit the network or reside in database nodes.
- Canvas-rendered, downloadable boarding passes that double as offline identification tokens.
- Zero-installation camera scanning leveraging the device's native video stream to decode QR payloads.

- An independent Node.js cron microservice that constantly evaluates event deadlines, queue entries, and qualification updates to dispatch targeted Web Push notifications.

1.10 KEY FEATURES

- **Cryptographic Master Authorization:** Event creation is restricted by a secret approval key (`VITE_APPROVAL_KEY`), preventing unauthorized event creation on the public domain.
- **Configurable Event Settings:** Dynamically specify event dates, registration closing deadlines, minimum/maximum team sizes, total days (\$N\$), current active day, total rounds (\$M\$), and current active round.
- **Dynamic Team Roster Registration:** Allows leaders to specify their details and dynamically add/remove team members with intuitive "Same college as leader" and "Same branch as leader" replication toggles.
- **Atomic Capacity Locking:** Utilizes Firebase Realtime Database transactions (`runTransaction`) on the `currentTeams` counter to mathematically prevent over-subscription beyond `maxTeams` .
- **Digital Boarding Pass Generation:** Renders a luxury airline-style boarding pass complete with flight cutouts, barcode styling, and metadata, exportable as a high-resolution PNG using HTML5 canvas rendering.
- **Dual Optical / Manual Attendance Scanner:** Decodes QR tokens via camera video feed at 10 FPS or allows manual alphanumeric Team ID input, rendering member checklists with haptic pulse verification.
- **Duplicate Check-in Guard:** Instantly alerts operators if a team attempts to check in more than once on the active day, preventing proxy attendance.
- **Multi-Round Qualification Manager:** Organizers toggle qualifications per round with visual star indicators and automatic finalists filtering.
- **Interactive Podium & Leaderboard:** Displays live rankings with podium reveals, gold/silver/bronze medals, and a GTA V-themed "WASTED" full-screen overlay when an eliminated team is queried.
- **Automated Cron Push Engine:** Dispatches 24-hour and 1-hour deadline reminders, status alerts, qualification announcements, and custom organizer broadcasts.
- **Comprehensive CSV Export Engine:** Generates sanitized, quoted CSV files for all team details, specific round qualifiers, and specific day attendees.

1.11 ADVANTAGES OF THE PROPOSED SYSTEM

1. **Zero Deployment Barrier for Users:** Runs entirely in the web browser; neither organizers nor participants need to install mobile apps from app stores.
2. **Sub-Second Real-Time Synchronization:** Changes made on the organizer dashboard reflect instantly across all participant leaderboards and tickets via Firebase WebSocket listeners.
3. **Robust Anti-Cheat and Proxy Prevention:** Unique QR payloads combining event identifiers and team codes prevent cross-event ticket reuse and duplicate check-in fraud.
4. **High Aesthetic and Ergonomic Polish:** Tailored with a custom dark-mode design system, gold accents, smooth typography, and haptic feedback on compatible touch devices.
5. **Fault Tolerant Architecture:** Client-side database calls are wrapped in an exponential backoff retry mechanism (`withRetry`), surviving momentary network drops.

1.12 LIMITATIONS OF THE PROJECT

As implemented in the current codebase, the system exhibits several real-world constraints:

- **Camera Dependency:** Optical attendance scanning requires a functional web camera with browser permissions; low-light environments may require manual Team ID fallback.
- **Client-Side Session Storage:** Organizer authentication state is maintained in browser `sessionStorage`; closing the browser tab terminates the administrative session.
- **Platform Web Push Constraints:** iOS devices require iOS 16.4+ and that the web app be added to the Home Screen to receive Web Push notifications.
- **Single Master Approval Key:** The current implementation utilizes a static master approval key rather than individual multi-tier administrator user accounts.

1.13 ORGANIZATION OF THE REPORT

This project report is organized into 18 structured chapters:

- **Chapter 1** presents the project background, problem statement, objectives, and scope.
- **Chapter 2** reviews existing systems, literature, and technology gaps.
- **Chapter 3** defines functional, non-functional, hardware, and software requirements.
- **Chapter 4** analyzes system architecture, data flows, and component interactions.
- **Chapter 5** details the complete technology stack and library selections.
- **Chapter 6** details the technical design of modules, components, and fault tolerance.
- **Chapter 7** provides the database schema, data dictionary, and ER diagrams.
- **Chapter 8** explains backend development, cron workers, and microservices.
- **Chapter 9** covers frontend architecture, pages, components, and haptics.
- **Chapter 10** elaborates on core module logic and algorithmic pseudocode.
- **Chapter 11** documents all HTTP APIs and database data contracts.
- **Chapter 12** evaluates security implementations, hashing, and access control.
- **Chapter 13** outlines testing strategies and presents an exhaustive test case table.
- **Chapter 14** discusses results, visual artifacts, and performance observations.
- **Chapter 15** explains production deployment on Firebase Hosting and Render.
- **Chapter 16** honestly articulates current technical and architectural limitations.
- **Chapter 17** proposes realistic future enhancements and research directions.
- **Chapter 18** concludes the report with a summary of achievements and learnings.
- **References & Appendices** provide verified sources, code listings, schemas, and manuals.

CHAPTER 2 — LITERATURE REVIEW / EXISTING SYSTEM

2.1 OVERVIEW OF EXISTING EVENT MANAGEMENT SYSTEMS

Event management in collegiate and technical environments has evolved through three historical paradigms:

1. **Manual Paper-Based Procedures:** Roster forms distributed in print, signed by team captains, and manually filed. Attendance is taken using printed sign-in rosters.
2. **Ad-Hoc Digital Toolchains (Spreadsheet Paradigm):** Solicit registrations via Google Forms, aggregate submissions in Google Sheets, distribute tickets or notices manually via bulk email or instant messaging (WhatsApp/Telegram), and check participants in by marking spreadsheet rows or paper prints.
3. **Commercial Event Platforms (Commercial SaaS):** Centralized platforms such as Eventbrite, Meetup, and Ticketmaster, or specialized competitive portals such as Devfolio and Unstop.

2.2 PROBLEMS IN CURRENT OPERATIONAL SYSTEMS

A rigorous technical examination reveals critical deficiencies across existing paradigms:

- **Data Fragmentation:** In the ad-hoc spreadsheet paradigm, there is no single source of truth. Changes to a spreadsheet do not propagate to participant badges or notification lists, resulting in desynchronized records.
- **Concurrency Conflicts:** When registrations surge prior to a deadline, concurrent submissions to standard forms often bypass capacity caps, resulting in over-registration and organizational embarrassment.
- **Vulnerability to Proxy Attendance:** Paper rosters and static spreadsheet check-ins cannot verify participant identity. One attendee can easily sign on behalf of absent teammates.
- **Absence of Multi-Day Roster Tracking:** Standard platforms record attendance as an isolated, binary event at the gate. They possess no built-in data structure to track attendance per day across multi-day conventions.
- **Inability to Model Multi-Round Progression:** Standard ticketing software cannot manage tournament rounds. Once a ticket is scanned, the platform considers its lifecycle complete. Organizers cannot qualify, eliminate, or re-verify teams for Stage 2 or Stage 3.
- **Notification Dilution:** Email notifications suffer from deliverability and open-rate issues (often landing in spam folders). Instant messaging groups suffer from notification muting and information burial under chat clutter.

2.3 EXISTING TECHNOLOGIES AND APPROACHES

Several technologies have been leveraged in modern event tooling:

- **Barcode & QR Systems:** Widely used in air travel and commercial cinema ticketing. However, commercial QR solutions require proprietary barcode hardware scanners and closed enterprise servers.
- **NFC / RFID Badging:** Deployed at high-budget corporate expos. While highly effective, NFC badging requires specialized hardware cards, writers, and readers, making it economically unfeasible for college budgets.

- **Web Push Notifications (W3C Push API):** An emerging open standard that allows background workers to deliver system notifications to desktop and mobile browsers without requiring native application installation.

2.4 ANALYSIS OF RELATED SYSTEMS

2.4.1 Google Forms & Google Sheets Toolchain

- *Strengths:* Free, ubiquitous, zero learning curve for form creation.
- *Weaknesses:* No automated ticket or QR code generation, no built-in camera scanner, no native push notifications, vulnerable to race conditions on registration caps, manual attendance checking.

2.4.2 Eventbrite / Commercial Ticketing SaaS

- *Strengths:* Highly polished ticketing, payment processing, mature mobile apps.
- *Weaknesses:* Optimized for single-attendee concert tickets, rigid fee structures, lacks collegiate team constructs (Team Leader, Members, College, Branch, Roll Number), no multi-round qualification engine, no multi-day member attendance granularity.

2.4.3 Devfolio / Unstop

- *Strengths:* Built specifically for hackathons and academic challenges; supports team formation.
- *Weaknesses:* Heavyweight, closed ecosystem requiring institutional onboarding, inflexible check-in scanning tools, lack of integrated progressive multi-day roll-call scanner with member checkboxes, no automated local cron push server.

2.5 COMPARATIVE ANALYSIS TABLE

Feature / Metric	Manual Sheets	Google Forms + Sheets	Eventbrite	Devfolio	EVENTRA (Proposed)
Deployment Model	Physical paper	Cloud office suite	Commercial SaaS	Specialized Portal	Decoupled Modern Web SPA
Cost to Institution	Paper printing	Free	High per-ticket fee	Contract / Platform fee	100% Free / Open Stack
Team Hierarchy Support	Manual notation	Flat form fields	None (individual)	Yes	Full (Leader + N Members)
Atomic Capacity Locking	Impossible	No (race condition)	Yes	Yes	Yes (<code>runTransaction</code>)
Digital Ticket Issuance	Physical badges	Manual add-ons	Email PDF	App QR code	Instant Boarding Pass PNG
Optical QR Scanning	None	Requires 3rd party	Mobile app	Proprietary tool	Browser Native (HTML5-QR)
Multi-Day Attendance	Paper sheets	Disjointed columns	No	No	Granular Per-Day Matrix
Member-Level Check-in	Manual tick	Manual tick	No (team level)	No (team level)	Interactive Checkbox Toggle
Multi-Round Filtering	Manual lists	Manual filters	No	Partial (manual)	Interactive Round Stepper
Podium & Rankings	Noticeboard	Spreadsheet	No	Website list	Gamified Podium & Reveal
Elimination Feedback	Verbal	Verbal	None	None	GTA V "WASTED" Audio/Visual
Push Notifications	None	None	Email only	Mobile app push	W3C Web Push (FCM + Cron)
Deduplication Engine	Manual	None	Internal	Internal	<code>notificationLog</code> Key Hash
Export Formats	Manual typing	Native sheet	CSV / XLS	CSV	Filtered Dynamic CSVs

2.6 RESEARCH AND TECHNOLOGICAL GAP

The literature and market survey reveals a distinct gap: **No existing open-source or lightweight web system combines collegiate team roster registration, atomic capacity limits, browser-based optical QR scanning, multi-day member-level roll-calls, multi-round qualification state machines, and autonomous background push notifications into a unified, zero-install Single Page Application.**

2.7 PROPOSED IMPROVEMENTS IN EVENTRA

Eventra bridges this gap by introducing:

1. An integrated, zero-configuration web architecture utilizing modern React 19 and Firebase RTDB.
2. Direct client-side ticket rendering via `html2canvas` and `qrcode.react`, allowing participants to save tickets as offline images.
3. Progressive optical decoding utilizing standard laptop webcams or mobile smartphone cameras without requiring native app installs.
4. An arbitrary $\$N$ -day attendance tracking model paired with an arbitrary $\$M$ -round competitive elimination workflow.
5. An automated Node.js cron microservice that continuously watches RTDB nodes and issues targeted push messages with idempotent deduplication keys.

CHAPTER 3 — REQUIREMENT ANALYSIS

3.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the core operational behaviors, features, and calculations executed by Eventra:

- **FR-01: Master Authorization & Event Creation:** The system must require a valid master approval key (`VITE_APPROVAL_KEY`) before permitting the creation of any new event. Upon verification, the organizer specifies a unique alphanumeric `eventId` and password.
- **FR-02: Cryptographic Password Hashing:** The system must hash organizer passwords client-side using the Web Crypto API (SHA-256) before transmission to the database. Plain-text passwords must never be stored.
- **FR-03: Event Parameter Configuration:** Organizers must be able to configure: Event Name, Description, Venue/Mode, Date & Time, Registration Deadline, Minimum Team Size, Maximum Team Size, Registration Open/Closed toggle, Maximum Team Capacity, Number of Event Days ($\$N$), Active Event Day, Number of Competition Rounds ($\$M$), Active Competition Round, and optional Payment Link.
- **FR-04: Atomic Capacity Allocation:** When a participant registers, the system must execute an atomic transaction on the event's `currentTeams` counter. If `currentTeams` meets or exceeds `maxTeams`, the registration must be rejected atomically.
- **FR-05: Team Registration with Leader & Member Hierarchy:** The system must collect team name, leader name, optional leader email, roll number, college, and branch. It must dynamically permit the addition of members up to the configured maximum, providing toggle shortcuts to inherit the leader's college and branch.
- **FR-06: Unique Team ID & QR Token Generation:** The system must generate a deterministic, padded Team ID in the format `{eventId}-T{XX}` (e.g., `hackathon2026-T01`) and encode the optical token string `{eventId}|{teamId}` into a high-density 2D QR code.

- **FR-07: Boarding Pass & Visual Ticket Virtualization:** The system must render a vintage-tech boarding pass ticket containing event metadata, passenger name, captain name, departure date, and QR stub. The ticket must be downloadable as a PNG file via HTML5 canvas virtualization.
- **FR-08: Optical Camera-Based QR Attendance Scanning:** The organizer interface must access the device's web camera, decode scanned QR codes at 10 FPS, parse the `{eventId}|{teamId}` payload, and query the database in real time.
- **FR-09: Multi-Day Member-Level Attendance Recording:** When a team QR code is scanned, the system must display all team members with active checkboxes, defaulting to present. Upon confirmation, the system must record attendance under the active day node (`dayAttendance/{currentDay}`) with millisecond timestamps.
- **FR-10: Duplicate Attendance Rejection:** If a team has already been marked present for the active day, the scanner must reject the check-in attempt, trigger a warning haptic/visual alert, and display previous check-in details.
- **FR-11: Multi-Round Competition Qualification:** Organizers must be able to view eligible teams for each round (\$1\$ to \$M\$) and toggle qualification status. In the final round (\$M\$), organizers must be able to assign podium positions (1st, 2nd, 3rd place) with mutual exclusivity enforcement.
- **FR-12: Real-Time Public Leaderboard & Gamified Elimination:** The system must provide a public leaderboard displaying team ranks, qualification pills, and round progression dots. When an eliminated team is searched, the system must trigger a GTA V-inspired "WASTED" audio-visual overlay. When winners are declared, an interactive reveal button must trigger celebration confetti.
- **FR-13: Push Notification Subscription & Categorization:** The system must prompt visitors and registrants for notification permissions via the W3C Push API and FCM. Tokens must be categorized under `fcmTokens/visitors` or associated directly with registered teams under `fcmTokens/teams/{eventId}/{teamCode}`.
- **FR-14: Scheduled Background Notification Dispatch:** The notification server must run cron jobs to evaluate registration deadlines (sending alerts at 24 hours and 1 hour remaining), detect registration status toggles, detect round qualification changes, and detect winner placements.
- **FR-15: Filtered Analytical CSV Export:** The system must generate and trigger browser downloads of sanitized CSV files for: (a) All Event Details, (b) Round-Qualified Teams for any round \$R\$, and (c) Present Teams for any day \$D\$.

3.2 NON-FUNCTIONAL REQUIREMENTS

- **NFR-01: Performance and Response Latency:** Client UI interactions must render within 100 ms. Real-time database listener updates must propagate across connected clients in under 500 ms under standard broadband conditions.
- **NFR-02: Optical Scanning Throughput:** The camera QR scanner must process and decode frames at a minimum rate of 10 frames per second (FPS), resolving valid tickets in under 1 second.
- **NFR-03: Security & Session Isolation:** Administrative routes (`/event-details`, `/dashboard`, `/scan`) must be protected by an authentication guard that verifies active `sessionStorage` tokens against the route's `eventId`. Cross-event access must be denied.
- **NFR-04: Resilience & Retry Logic:** All asynchronous database writes and reads must pass through a recursive exponential backoff mechanism (`withRetry`), retrying transient network failures up to 2 times before throwing an error.

- **NFR-05: Idempotence in Notifications:** The notification server must guarantee that duplicate notifications are never dispatched for the same event deadline or qualification update by maintaining an idempotent deduplication hash log (`notificationLog`).
- **NFR-06: Cross-Browser Compatibility:** The frontend must function consistently across Chromium-based browsers (Chrome, Edge, Brave), Mozilla Firefox, and Apple Safari on both desktop and mobile form factors.
- **NFR-07: Usability and Visual Polish:** The user interface must employ a coherent design language with high contrast, legible typography (JetBrains Mono, Crimson Pro), ambient background lighting, and responsive layouts across screens from 360px to 4K.

3.3 HARDWARE REQUIREMENTS

3.3.1 Client / End-User Devices

- **Processor:** Dual-Core 1.8 GHz or higher (Intel Core i3/AMD Ryzen 3 or ARM equivalent on mobile).
- **RAM:** Minimum 2 GB (4 GB recommended for smooth camera canvas stream processing).
- **Display:** Minimum resolution of \$360 \times 640\$ pixels for mobile; \$1280 \times 720\$ pixels for organizer dashboard.
- **Peripherals:** Integrated or USB web camera capable of capturing video at 720p @ 15 FPS minimum (required only for scanning check-in desks).

3.3.2 Server / Hosting Environment

- **Notification Server Runtime:** Node.js runtime environment with minimum 512 MB RAM and 0.5 vCPU (standard free/starter tier on Render or equivalent PaaS).
- **Database Infrastructure:** Google Cloud Firebase Realtime Database managed infrastructure.

3.4 SOFTWARE REQUIREMENTS

- **Client Operating System:** Windows 10/11, macOS 11+, Linux Ubuntu 20.04+, Android 10+, or iOS 16.4+.
- **Web Browser:** Google Chrome 110+, Microsoft Edge 110+, Mozilla Firefox 115+, or Apple Safari 16.4+.
- **Server Runtime:** Node.js version 18.0.0 or higher.
- **Package Manager:** npm version 9.0.0 or higher.
- **Version Control:** Git 2.34+.

3.5 DEVELOPMENT ENVIRONMENT

- **Primary IDE:** Visual Studio Code / Antigravity IDE.
- **Terminal & Shell:** PowerShell 7 on Windows / Bash on Linux.
- **Bundler & Build Tool:** Vite 8.0.4.
- **Compiler:** TypeScript 6.0.2 with `tsconfig.app.json` and `tsconfig.node.json` configurations.

- **Linters:** ESLint 9.39.4 with React Hooks and TypeScript plugins.

3.6 USER (PARTICIPANT) REQUIREMENTS

- Access event details, guidelines, and rules via a shared link without creating an account.
- Complete registration forms with instantaneous validation of member limits.
- Save, view, and screenshot an airline-style digital boarding pass with an embedded QR code.
- Query their team status on the public leaderboard and receive visual confirmation of qualification or elimination.
- Opt into web push notifications with a single click to receive real-time schedule and qualification alerts.

3.7 ADMIN (ORGANIZER) REQUIREMENTS

- Securely provision an event namespace using the institutional master approval key.
- Authenticate into the event-specific dashboard using hashed credentials.
- Adjust event parameters (dates, deadlines, day counts, round counts) at any point during event execution.
- Utilize a camera-based scanner to check in teams and record attendance per member per day.
- Advance qualifying teams across rounds and designate podium champions.
- Dispatch targeted push notifications to all teams, qualified round teams, or specific selected teams.
- Download verified attendance and qualification reports in Comma-Separated Values (CSV) format.

3.8 SYSTEM CONSTRAINTS

- **Network Dependency:** Firebase Realtime Database operations require an active internet connection; while `withRetry` handles temporary drops, prolonged offline operation is not supported.
- **Single Master Key:** All events are provisioned using a single institutional master key (`VITE_APPROVAL_KEY`) embedded in the environment configuration.
- **Camera Lighting & Focus:** Optical QR decoding depends on ambient lighting conditions and physical camera focus quality.

3.9 OPERATIONAL ASSUMPTIONS

- Organizers possess a modern smartphone or laptop with an operational camera for attendance scanning.
- Participants have internet access during registration and leaderboard queries.
- The hosting cloud provider (Firebase/Render) maintains an operational SLA $\geq 99.5\%$.

3.10 FEASIBILITY STUDY

3.10.1 Technical Feasibility

The project utilizes proven, industry-standard technologies: React 19, Vite, TypeScript, Express, and Firebase. The optical QR scanner utilizes the well-established `html5-qrcode` library wrapping the native HTML5 Canvas and MediaDevices APIs. Push notifications leverage the W3C Push API and Firebase Cloud Messaging standard. All required APIs are mature, heavily documented, and natively supported across modern browsers. The project is technically viable.

3.10.2 Economic Feasibility

Eventra is engineered to operate entirely within free-tier cloud architectures during typical academic usage:

- **Frontend Hosting:** Firebase Hosting provides a generous free tier (10 GB storage, 360 MB/day transfer), easily supporting an academic symposium.
- **Database:** Firebase Realtime Database free Spark plan supports 1 GB stored data, 10 GB/month transfer, and 100 simultaneous connections.
- **Notification Server:** Hosted on the free tier of cloud PaaS providers (such as Render), running on 512 MB containers.
- **Zero Software Licensing Costs:** All frameworks, compilers, icons, and libraries are open-source under MIT or Apache 2.0 licenses. The economic feasibility is absolute.

3.10.3 Operational Feasibility

The operational workflow requires minimal user training. The participant registration form mirrors standard web forms. The optical scanner requires only pointing the device's camera at a printed or mobile screen QR code. The organizer dashboard provides intuitive tabbed interfaces with visual feedback, status badges, and haptic confirmations. The project is operationally feasible and intuitive.

CHAPTER 4 — SYSTEM ANALYSIS AND ARCHITECTURE

4.1 SYSTEM OVERVIEW

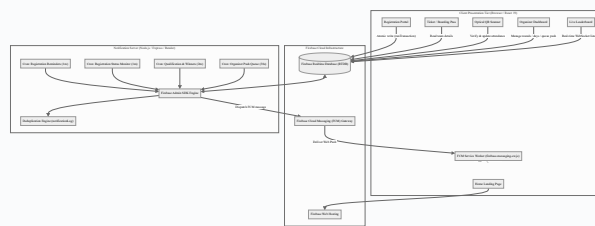
Eventra is architected as a distributed, decoupled client-server system comprising three primary architectural layers:

1. **Client Presentation Tier (SPA Frontend):** Developed with React 19, TypeScript, and Vite, delivering client-side routing, optical barcode decoding, canvas virtualization, and responsive UI components.
2. **Persistence and Realtime Synchronization Tier (Firebase RTDB):** A managed cloud NoSQL database that synchronizes application state as a JSON document tree across all subscribed clients via bi-directional WebSockets.

3. **Autonomous Background Worker Tier (Notification Microservice):** An Express and Node-cron service operating with Firebase Admin credentials, periodically inspecting database state, managing notification queues, and dispatching Web Push payloads via the FCM Gateway.

4.2 HIGH-LEVEL SYSTEM ARCHITECTURE

The high-level architecture separates user-driven read/write transactions from asynchronous background evaluation tasks. Participants and organizers interact directly with the client tier, which communicates with the Firebase Realtime Database over secure WebSockets (WSS). Concurrently, the independent Notification Server microservice establishes an administrative channel to the database, tracking scheduled deadlines and draining organizer broadcast queues.



4.3 COMPONENT-LEVEL ARCHITECTURE

At the component level, Eventra is organized into functional modules with clear separation of concerns:

- **Presentation Components:** Reusable interface widgets (`Button`, `Input`, `Textarea`, `GlassCard`, `StatusBadge`, `LoadingSpinner`, `Skeleton`).
- **Domain Components:** Business-logic components (`TicketCard`, `QRCodeDisplay`, `QRScanner`, `CSVDownloadModal`, `NotificationPanel`, `NotificationPrompt`).
- **State Management & Routing:** `AuthContext` providing session state, `ProtectedRoute` preventing unauthorized navigation, and `BrowserRouter` handling client-side URLs.
- **Utility Subsystems:** `fcm.ts` (push client), `haptics.ts` (hardware vibration), `db-retry.ts` (fault tolerance), and `utils.ts` (cryptographic hashing, CSV formatting).

4.4 FRONTEND ARCHITECTURAL FLOW

When a user navigates to an Eventra route:

1. `main.tsx` mounts the application wrapped in `AuthProvider`.
2. `AuthContext` initializes by reading cached organizer sessions from browser `sessionStorage`.
3. `App.tsx` configures the routing table. Public routes (`/`, `/create-event`, `/organizer-login`, `/register/:eventId`, `/ticket/:eventId/:teamId`, `/leaderboard/:eventId`) render directly.
4. Protected routes (`/event-details/:eventId`, `/dashboard/:eventId`, `/scan/:eventId`) pass through `ProtectedRoute`. If the session is missing or the authenticated `eventId` does not match the URL parameter, the route redirects to `/organizer-login`.

5. Pages establish active listeners on specific database paths using the Firebase Web SDK (`ref` , `get` , `onValue`), keeping the UI synchronized with zero page reloads.

4.5 BACKEND & NOTIFICATION SERVER ARCHITECTURE

The backend operates as an event-driven scheduler. Rather than requiring HTTP polling from the client, the backend autonomously evaluates database state using `node-cron` :

- **Every 30 seconds:** Scans `notificationQueue/{eventId}` for manual broadcasts queued by organizers. Upon detection, it queries target device tokens and transmits FCM messages.
- **Every 60 seconds:** Evaluates event registration deadlines, triggering 24-hour and 1-hour countdown push notifications to registered teams.
- **Every 60 seconds:** Detects registration status transitions (`registrationOpen` toggles or deadline extensions) and broadcasts schedule updates.
- **Every 120 seconds:** Compares in-memory qualification caches against RTDB states to identify newly qualified teams or podium winners, dispatching congratulatory push alerts.
- **Daily at 3:00 AM:** Purges deduplication records older than 7 days from `notificationLog` .

4.6 DATABASE SYNCHRONIZATION TOPOLOGY

Eventra uses a hierarchical JSON document store in Firebase Realtime Database. The database topology operates via persistent WebSockets over TLS:

- When an organizer scans a QR code, an `update()` operation writes to `events/{eventId}/teams/{teamCode}/dayAttendance/{currentDay}` .
- Firebase servers push the delta payload to all connected clients listening on `events/{eventId}/teams` .
- The live dashboard immediately increments the "Day \$\$\$ Present" counter, updates the attendance progress bar, and alters the status badge from red ("Absent") to green ("Present") without manual page refreshing.

4.7 API ARCHITECTURE AND DATA CONTRACTS

Communication between client, database, and backend follows strict schema contracts:

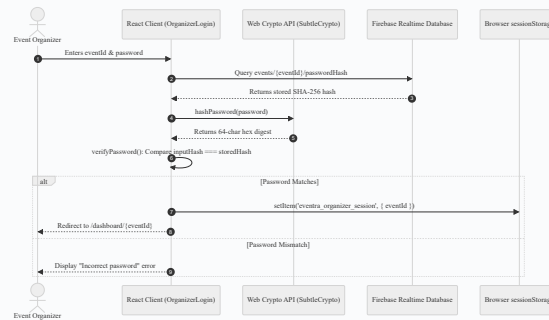
- **Client to RTDB:** Direct Firebase Web SDK operations (`get` , `set` , `update` , `runTransaction`).
- **Client to Backend:** Optional HTTP REST endpoints (`POST /notify`) using JSON payloads.
- **Backend to RTDB:** Administrative privileged queries via Firebase Admin SDK.
- **Backend to FCM:** HTTP/2 multiplexed socket connection to Google Cloud Messaging servers.

4.8 AUTHENTICATION & CRYPTOGRAPHIC SESSION MODEL

Eventra implements an event-scoped cryptographic authentication architecture:

1. During event creation, the organizer inputs a master `approvalKey` and an event password.
2. The client checks `approvalKey === VITE_APPROVAL_KEY` .

3. The password is fed to `crypto.subtle.digest('SHA-256', data)` using the Web Crypto API, producing a 64-character hexadecimal digest.
4. Only this SHA-256 digest is persisted under `events/{eventId}/passwordHash`.
5. During login, the entered password is hashed on the client and compared against the stored hash via `verifyPassword()`.
6. Upon successful verification, `login(eventId)` writes `{ eventId }` into `sessionStorage` under the key `eventra_organizer_session`.



4.9 AUTHORIZATION MATRIX

Authorization is enforced via client-side router guards and database key boundaries:

Resource / Route	Public Visitor	Registered Participant	Authenticated Organizer
Landing Page (/)	Full Access	Full Access	Full Access
Create Event (/create-event)	Protected by Approval Key	Protected by Approval Key	Full Access
Team Registration (/register/:id)	Full Access	Full Access	Full Access
View Ticket (/ticket/:id:team)	Full Access (Direct Link)	Full Access (Direct Link)	Full Access
Leaderboard (/leaderboard/:id)	Full Access	Full Access	Full Access
Event Settings (/event-details/:id)	Access Denied	Access Denied	Authenticated Organizer Only
Dashboard (/dashboard/:id)	Access Denied	Access Denied	Authenticated Organizer Only
QR Scanner (/scan/:id)	Access Denied	Access Denied	Authenticated Organizer Only
Database Write (events/{id}/teams)	Registration Write	Registration Write	Full Administrative Access

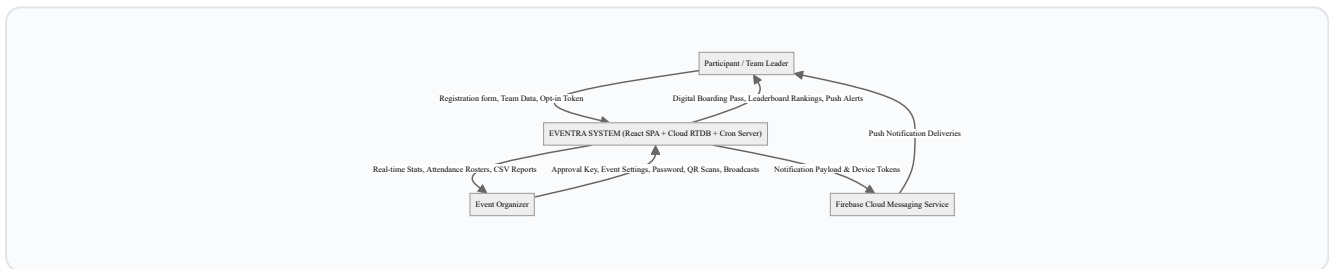
4.10 REQUEST-RESPONSE PROCESSING PIPELINE

For transactional operations (such as team registration):

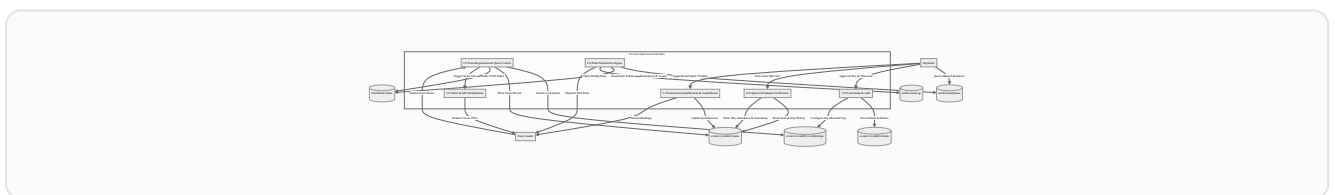
1. **Client Validation:** Form inputs are validated against schema rules (email regex, team size min/max).
2. **Duplicate Email Verification:** The sanitized email key (`formatEmailForDb`) is checked under `events/{eventId}/registeredEmails/{emailKey}` .
3. **Atomic Counter Lock:** A transaction is dispatched to `events/{eventId}/eventSettings/currentTeams` . The database engine increments the counter atomically only if `count < maxTeams` .
4. **Team Record Provisioning:** With the returned counter value, the deterministic Team ID (`{eventId}-T{XX}`) is generated, and full team metadata is written.
5. **Token Association:** If the user opted into notifications, the device's FCM registration token is indexed under `fcmTokens/teams/{eventId}/{teamCode}` .
6. **Navigation:** Client redirects to the confirmation page (`/registration-success`).

4.11 END-TO-END DATA FLOW DIAGRAMS

4.11.1 Level 0 DFD (Context Diagram)



4.11.2 Level 1 DFD (Decomposed Functional Modules)



4.12 OVERALL SYSTEM OPERATIONAL WORKFLOW

The overall operational lifecycle of an event in Eventra proceeds through five sequential phases:

1. **Provisioning Phase:** The organizer authenticates with `APPROVAL_KEY` , sets the event identifier, defines password, and configures event schedule, days, and rounds.
2. **Registration Phase:** The registration link is shared. Participants register their teams. Quotas are enforced atomically. Boarding passes are generated and saved.

3. **Check-In Phase:** On event day, venue staff open `/scan/:eventId` on mobile or laptop browsers, scanning physical or screen-rendered QR codes. Attendance is recorded per member for the active day.
 4. **Competition Phase:** As stages conclude, the organizer navigates to Dashboard $\rightarrow$ Qualified tab, advancing teams for the next round. The public leaderboard updates in real time. Eliminated team searches trigger the GTA V "WASTED" overlay.
 5. **Culmination & Reporting Phase:** Final podium rankings are assigned. Confetti celebration is revealed on the public leaderboard. The organizer downloads verified CSV analytical reports.
-

CHAPTER 5 — TECHNOLOGY STACK

5.1 CORE PROGRAMMING LANGUAGES

- **TypeScript (version ~6.0.2):** Primary programming language for the entire client application. TypeScript provides static typing, interface contracts (`types/index.ts`), compile-time error detection, and code refactoring safety.
- **JavaScript (ES2022 / Node.js CommonJS):** Utilized in the `notification-server` microservice for straightforward, dependency-free execution in Node.js runtime environments.
- **HTML5:** Semantic document markup, native `<canvas>` rendering for QR and ticket virtualization, and camera media capture.
- **CSS3:** Modern layout specifications including Flexbox, CSS Grid, custom properties (CSS variables), keyframe animations, and backdrop filters.

5.2 FRONTEND UI FRAMEWORK & RUNTIME

- **React (version ^19.2.4):** The core UI library. React 19 provides reactive component rendering, hooks (`useState` , `useEffect` , `useCallback` , `useRef`), and high-speed DOM reconciliation.
- **React DOM (version ^19.2.4):** Client-side DOM renderer executing within the browser root element (`createRoot`).
- **Vite (version ^8.0.4):** Next-generation frontend tooling and build orchestrator. Vite leverages native ES modules during development for instant Hot Module Replacement (HMR) and utilizes Rollup for optimized production bundling.
- **React Router DOM (version ^7.14.1):** Client-side routing engine managing URL navigation, dynamic path parameters (`:eventId` , `:teamId`), and declarative route protection (`ProtectedRoute`).

5.3 STYLING SYSTEM AND DESIGN PHILOSOPHY

- **Tailwind CSS (version ^4.2.2):** Integrated via `@tailwindcss/vite` . Provides utility-first styling for structural layout and spacing.
- **Eventra Design System (`src/styles/eventra-shared.css` & `src/index.css`):** A custom-crafted vintage-tech and luxury dark aesthetic utilizing:

- *Color Palette*: Background `#0a0a0f`, Surface `#1A1A1A`, Surface-2 `#242424`, Primary Gold `#C6A969`, Accent Gold `#D4AF37`, Success Green `#4ADE80`, Danger Red `#F87171`, Warning Yellow `#FBBF24`.
- *Typography*: Google Fonts `Crimson Pro` (editorial serif for headings), `JetBrains Mono` (technical monospaced for metadata and data fields), and `DM Sans` / `Playfair Display`.
- *Visual Effects*: Ambient radial glow orbs, subtle 44px grid patterns, glassmorphism cards with `backdrop-filter: blur(16px)`, and gold-trimmed borders.

5.4 DATABASE TECHNOLOGY

- **Firestore Database (Google Cloud)**: Cloud-hosted NoSQL document database. Data is stored as a native JSON hierarchy and synchronized in real time across all connected clients via WebSockets over TLS. Chosen over traditional relational databases for its zero-server setup, millisecond data propagation, offline caching, and atomic transaction primitives.
- **Firestore Admin SDK (version ^12.12.0)**: Client-side library (`firebase/app`, `firebase/database`, `firebase/messaging`) used to interface directly with Firestore services from the React application.

5.5 BACKEND SERVER RUNTIME & MICROSERVICES

- **Node.js (Engine >= 18.0.0)**: Server-side JavaScript runtime executing the notification microservice.
- **Express (version ^4.21.2)**: Minimalist HTTP server framework providing REST endpoints (`/`, `/status`, `/debug`, `/notify`) and CORS middleware.
- **Firestore Admin SDK (version ^13.0.2)**: Privileged server SDK providing administrative access to Firestore RTDB and the Firestore Cloud Messaging engine without client-side permission restrictions.
- **node-cron (version ^3.0.3)**: Pure JavaScript task scheduler executing recurring background cron expressions within the Node process.
- **cors (version ^2.8.5)**: Cross-Origin Resource Sharing middleware enabling secure API invocation from the client web origin.
- **dotenv (version ^16.4.7)**: Environment variable manager loading configuration from local `.env` files into `process.env`.

5.6 MESSAGING AND PUSH NOTIFICATION INFRASTRUCTURE

- **Firestore Cloud Messaging (FCM)**: Cross-platform messaging solution used to route push notifications to web clients.
- **W3C Web Push Protocol**: Industry standard utilizing public key cryptography (VAPID keys) to authenticate server push messages to user agents.
- **Service Worker (`public/firebase-messaging-sw.js`)**: Background browser worker script that intercepts push payloads when the web application is closed or minimized, rendering native operating system notifications.

5.7 PERIPHERAL AND AUXILIARY LIBRARIES

- **html5-qrcode (version ^2.3.8)**: Robust JavaScript library utilizing the browser's MediaDevices API to capture real-time camera video streams and decode 2D QR codes on the client CPU.
- **qrcode.react (version ^4.2.0)**: React component rendering crisp SVG and HTML5 canvas 2D QR codes using client-side algorithms.
- **html2canvas (version ^1.4.1)**: JavaScript library that reads the DOM structure of the boarding pass ticket and renders it into a downloadable HTML5 canvas element.
- **canvas-confetti (version ^1.9.4)**: High-performance confetti particle simulator used during leaderboard podium reveals.
- **lucide-react (version ^1.8.0)**: Iconography library providing lightweight SVG icons.
- **Web Vibration API (navigator.vibrate)**: Native browser hardware API wrapped in `src/lib/haptics.ts` to provide tactile feedback patterns.

5.8 BUILD PIPELINE AND PACKAGE MANAGEMENT

- **npm**: Package dependency resolution and script execution (`dev`, `build`, `lint`, `preview`).
- **TypeScript Compiler (tsc -b)**: Validates type safety across the application and checks configuration constraints before bundling.
- **Rollup (via Vite)**: Compiles TypeScript, tree-shakes dead code, minifies CSS/JS assets, and splits vendor chunks into the `/dist` directory.

5.9 HOSTING AND CLOUD DEPLOYMENT INFRASTRUCTURE

- **Firebase Hosting**: High-speed Content Delivery Network (CDN) providing SSL-secured global distribution of static frontend assets, with single-page application URL rewriting configured in `firebase.json`.
- **Render Cloud Application Platform**: Platform-as-a-Service (PaaS) executing the persistent Node.js notification microservice container.

CHAPTER 6 — SYSTEM DESIGN

6.1 ARCHITECTURAL DESIGN PRINCIPLES

Eventra is engineered around five fundamental software architecture principles:

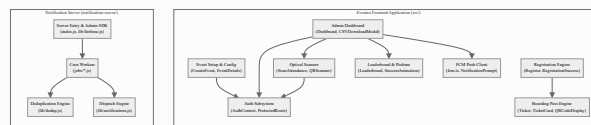
1. **Decoupled Responsibilities**: Client rendering, database synchronization, and background scheduled processing exist as independent subsystems.
2. **Offline Resilience & Graceful Degradation**: Core ticket viewing functions utilize local canvas caching. Database operations incorporate exponential backoff retry wrappers. Unsupported features (such as haptics

or push on older browsers) fail gracefully without breaking UI execution.

3. **Zero-Poll Reactive Synchronization:** Rather than issuing recurring HTTP GET requests, the client establishes bi-directional WebSocket subscriptions to RTDB nodes.
4. **Stateless Administrative Sessions:** Administrative authorization is maintained client-side in `sessionStorage` and validated route-by-route, eliminating heavy server-side session stores.
5. **Idempotent Background Workers:** Every scheduled notification worker computes a unique deterministic hash key before dispatching messages, guaranteeing zero duplicate alerts.

6.2 MODULE BREAKDOWN

The codebase is structured into cohesive modules:



6.3 COMPONENT HIERARCHY AND INTERACTION MODEL

The React component tree follows a clean, single-root layout structure:

- **Layout** : Renders the persistent ambient background, subtle animated grid lines, fixed `Navbar`, floating `NotificationPrompt` toast listener, and page footer.
- **Outlet** : Dynamically mounts the active page view depending on browser route.
- **Atomic Subcomponents**: Pages compose smaller widgets (`Input`, `Button`, `GlassCard`) that inherit styles from the shared design system.

6.4 CRYPTOGRAPHIC HASHING AND SECURITY SUBSYSTEM

Organizer passwords are never transmitted in cleartext or evaluated on a backend server. Instead, Eventra leverages the browser's native **Web Crypto API**:

```
const hash = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(password));
```

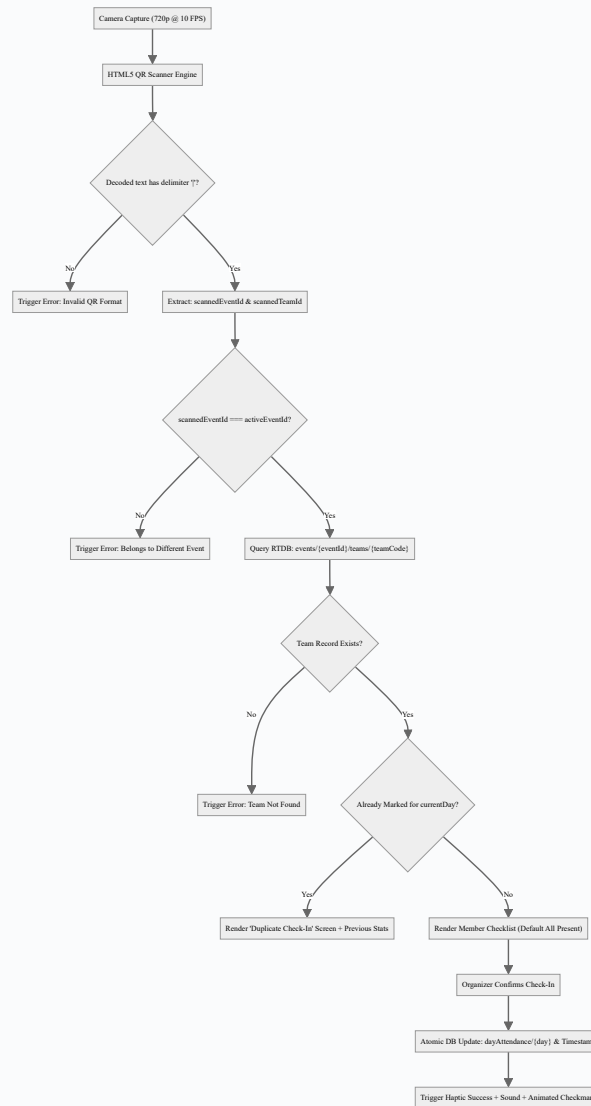
The resulting 256-bit hash buffer is converted into a 64-character lowercase hexadecimal string. During authentication, the same algorithm is applied to user input and verified using strict string equality.

6.5 QR ENCODING AND OPTICAL SCANNING PIPELINE

The ticketing pipeline encodes a pipe-delimited payload:

```
const QRPayload = `${eventId}|${teamId}`;
```

For example: `hackathon2026|hackathon2026-T01`.



6.6 MULTI-DAY ATTENDANCE STATE MACHINE

Unlike standard ticketing systems that treat attendance as a single boolean, Eventra implements an arbitrary $N \times N$ -day attendance matrix:

- Each team maintains a `dayAttendance` dictionary keyed by day index string ("1" , "2" , "3" , etc.).
- Each entry stores:

```

{
  "marked": true,
  "markedAt": 1740920000000,
  "members": [
    { "name": "Alice", "present": true, "rollNumber": "21CS01" },
    { "name": "Bob", "present": false, "rollNumber": "21CS02" }
  ]
}

```

- The organizer advances `currentDay` via the Event Details stepper. The scanner dynamically binds to `currentDay`. Day 1 attendance automatically mirrors to the legacy `attendanceMarked` boolean for backwards compatibility.

6.7 MULTI-ROUND QUALIFICATION AND ELIMINATION ENGINE

Eventra models multi-stage tournaments using an arbitrary M -round state machine:

- Under each team record, a `qualifications` map stores round states:

```
{
  "1": true,
  "2": true,
  "3": false
}
```

- **Round Advancement Logic:** In Round $R > 1$, the Dashboard displays only teams that were qualified in Round $R - 1$.
- **Final Round & Podium Mode:** In the final round (M), the UI transitions from qualification toggles to podium assignment medals: 1st Place (🥇), 2nd Place (🥈), 3rd Place (🥉). Assigning a position automatically clears it from any previous holder, enforcing strict uniqueness.

6.8 ASYNCHRONOUS NOTIFICATION QUEUE AND DEDUPLICATION

To enable manual broadcasts without direct FCM server credentials on the client, Eventra uses a decoupled queue pattern:

1. **Client Enqueues:** The organizer composes an announcement on the Dashboard. The client writes an entry into `notificationQueue/{eventId}/{pushId}` containing title, message body, target filter (`all_teams`, `qualified_round`, `winners`, `specific_teams`), and timestamp.
2. **Worker Drains:** Every 30 seconds, `organizer-push.js` queries unprocessed queue entries.
3. **Recipient Resolution:** The worker queries `events/{eventId}/teams` to resolve device tokens matching the target criteria.
4. **Execution & Marking:** The worker transmits messages via the FCM Admin SDK and updates the queue entry with `processed: true`, timestamp, and dispatched counts.
5. **Deduplication Engine:** Scheduled cron jobs compute an idempotent key: $\text{\texttt{\$}\text{\texttt{\$}}\text{\texttt{\text{dedupKey}}} = \text{\texttt{\text{eventId}}} \parallel \text{\texttt{"_"}} \parallel \text{\texttt{type}} \parallel \text{\texttt{"_"}} \parallel \text{\texttt{deadline/timestamp}}\text{\texttt{\$}}\text{\texttt{\$}}$ The worker checks `notificationLog/{dedupKey}` before sending. If the key exists, execution is skipped, completely eliminating duplicate notifications.

6.9 ERROR HANDLING AND FAULT TOLERANCE STRATEGY

Eventra incorporates fault tolerance at multiple levels:

- **`withRetry` Database Wrapper:** Wraps asynchronous operations in recursive exponential backoff:

```
// Retries failed reads/writes with incremental delay (delay * 1.5)
await withRetry(() => get(ref(db, path)), 2, 1000);
```

- **Camera Fallback:** If browser camera permissions are denied or hardware fails, an integrated manual alphanumeric lookup field allows instant check-in.
- **Offline Banner:** An event listener monitors `window.navigator.onLine`, rendering warning alerts when internet connectivity is severed.
- **Stale Token Eviction:** If the FCM gateway returns `registration-token-not-registered`, the notification server automatically scrubs the invalid token from the database, preventing wasted resources on subsequent runs.

6.10 SCALABILITY AND PERFORMANCE CONSIDERATIONS

- **Data Pruning:** Dedup logs older than 7 days are automatically removed by daily cron workers, preventing infinite database bloat.
 - **Lightweight Document Queries:** Targeted queries inspect specific child nodes (e.g., `events/{id}/details/eventName`) rather than loading monolithic branches.
 - **Client-Side Virtualization:** Static assets, ticket cards, and QR images are generated entirely within the client runtime using HTML5 Canvas, offloading server CPU loads.
-

CHAPTER 7 — DATABASE DESIGN

7.1 DATABASE PARADIGM AND TECHNOLOGY SELECTION

Eventra utilizes the **Firestore Realtime Database (RTDB)**, a cloud-hosted NoSQL document database. Unlike traditional relational database management systems (RDBMS) that require fixed table schemas, joins, and database migrations, RTDB stores all data as a single hierarchical JSON document tree.

RTDB was selected for this project due to four decisive architectural advantages:

1. **Bi-directional WebSocket Synchronization:** Clients do not poll; the database engine pushes deltas to listening clients with sub-500ms latency.
2. **Native Atomic Transaction API:** The `runTransaction()` method guarantees atomic updates on counters, crucial for eliminating race conditions during registration rushes.
3. **Zero Maintenance Infrastructure:** Fully managed by Google Cloud, eliminating database administration, connection pooling, and operating system patching.
4. **Hierarchical Document Modeling:** Naturally represents collegiate event hierarchies where teams, members, and day records nest logically within specific event nodes.

7.2 REALTIME DATABASE ARCHITECTURE

The database schema is organized under four top-level root nodes:

- `/events` : Contains all event namespaces, details, settings, and team records.
- `/fcmTokens` : Contains device push registration tokens indexed for fast broadcast retrieval.
- `/notificationQueue` : Holds asynchronous push broadcast requests queued by organizers.
- `/notificationLog` : Stores idempotent dispatch records preventing duplicate cron notifications.

7.3 COMPLETE NODE HIERARCHY AND DATA DICTIONARY

```
root
├── events
│   ├── {eventId}
│   │   ├── createdAt: number (ms since epoch)
│   │   ├── passwordHash: string (64-char hex SHA-256)
│   │   ├── teamCount: number (legacy total counter)
│   │   ├── registeredEmails
│   │   │   └── {sanitizedEmail}: true
│   │   ├── details
│   │   │   ├── eventName: string
│   │   │   ├── description: string
│   │   │   ├── dateTime: string (ISO / datetime-local)
│   │   │   ├── venue: string
│   │   │   ├── teamSizeMin: number
│   │   │   ├── teamSizeMax: number
│   │   │   └── paymentLink: string | null
│   │   └── eventSettings
│   │       ├── currentTeams: number (atomic counter)
│   │       ├── maxTeams: number | null
│   │       ├── registrationOpen: boolean
│   │       ├── registrationDeadline: string | null (ISO string)
│   │       ├── numberOfDays: number (total event days)
│   │       ├── currentDay: number (active day index)
│   │       ├── numberOfRounds: number (total competition rounds)
│   │       └── currentRound: number (active round index)
│   └── teams
│       ├── {teamCode} (e.g., "T01")
│       │   ├── teamId: string (e.g., "hackathon2026-T01")
│       │   ├── teamName: string
│       │   ├── leader: string
│       │   ├── email: string | null
│       │   ├── createdAt: number (ms timestamp)
│       │   ├── attendanceMarked: boolean (Day 1 legacy flag)
│       │   ├── position: number | null (1, 2, or 3 for winners)
│       │   ├── fcmToken: string | null
│       │   ├── fcmTokenUpdatedAt: number | null
│       │   ├── qualifications
│       │   │   └── {roundNumber}: boolean
│       │   ├── dayAttendance
│       │   │   └── {dayNumber}
│       │   │       ├── marked: boolean
│       │   │       ├── markedAt: number (ms timestamp)
│       │   │       └── members: Array<MemberAttendanceObject>
│       │   └── members: Array<MemberObject>
├── fcmTokens
│   ├── visitors
│   │   ├── {sanitizedTokenKey}
│   │   │   ├── token: string
│   │   │   ├── userAgent: string
│   │   │   ├── platform: string
│   │   │   ├── language: string
│   │   │   └── createdAt: object (serverTimestamp)
```

```

├── updatedAt: object (serverTimestamp)
├── teams
│   ├── {eventId}
│   │   ├── {teamCode}
│   │   │   ├── token: string
│   │   │   ├── teamId: string
│   │   │   ├── teamName: string
│   │   │   ├── leader: string
│   │   │   ├── email: string
│   │   │   ├── eventId: string
│   │   │   └── updatedAt: object (serverTimestamp)
├── notificationQueue
│   ├── {eventId}
│   │   ├── {pushId}
│   │   │   ├── title: string
│   │   │   ├── body: string
│   │   │   ├── target: "all_teams" | "qualified_round" | "winners" | "specific_teams"
│   │   │   ├── targetRound: number | null
│   │   │   ├── teamCodes: Array<string> | null
│   │   │   ├── url: string
│   │   │   ├── createdAt: number
│   │   │   ├── processed: boolean
│   │   │   ├── processedAt: number | null
│   │   │   └── result: { sent: number, failed: number }
├── notificationLog
│   ├── {sanitizedDedupKey}
│   │   ├── key: string
│   │   ├── sentAt: number
│   │   ├── type: string
│   │   ├── eventId: string
│   │   └── eventName: string

```

7.3.1 Data Dictionary: `events/{eventId}/details`

Field Name	Data Type	Nullable	Description / Constraints
<code>eventName</code>	String	No	Full display title of the event (e.g., "National Hackathon 2026").
<code>description</code>	String	No	Detailed explanation of event themes, guidelines, and rules.
<code>dateTime</code>	String	Yes	ISO string representation of the event scheduled commencement date and time.
<code>venue</code>	String	No	Physical campus hall, auditorium, lab number, or online meeting link.
<code>teamSizeMin</code>	Number	No	Minimum allowable team members including leader (Integer ≥ 1).
<code>teamSizeMax</code>	Number	No	Maximum allowable team members including leader (Integer $\geq$ teamSizeMin).
<code>paymentLink</code>	String	Yes	Optional URL to external payment gateway (e.g., Razorpay, UPI).

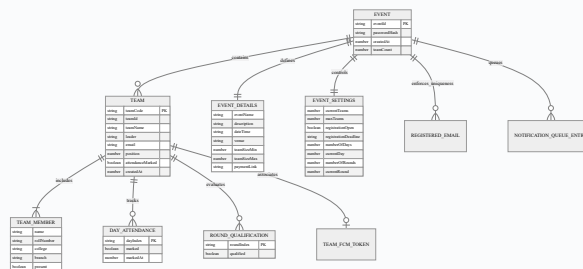
7.3.2 Data Dictionary: `events/{eventId}/eventSettings`

Field Name	Data Type	Nullable	Description / Constraints
<code>currentTeams</code>	Number	No	Total accepted teams counter, mutated strictly via <code>runTransaction()</code> .
<code>maxTeams</code>	Number	Yes	Upper bound capacity limit. If null, registrations are uncapped.
<code>registrationOpen</code>	Boolean	No	Administrative master switch to open or pause registration intake.
<code>registrationDeadline</code>	String	Yes	ISO timestamp when registration automatically closes.
<code>numberOfDays</code>	Number	No	Total days attendance will be tracked (Default 1, Integer ≥ 1).
<code>currentDay</code>	Number	No	Active day index ($1 \leq \text{currentDay} \leq \text{numberOfDays}$).
<code>numberOfRounds</code>	Number	No	Total competition stages for qualification (Default 1, Integer ≥ 1).
<code>currentRound</code>	Number	No	Active competition round ($1 \leq \text{currentRound} \leq \text{numberOfRounds}$).

7.3.3 Data Dictionary: `events/{eventId}/teams/{teamCode}`

Field Name	Data Type	Nullable	Description / Constraints
<code>teamId</code>	String	No	Unique deterministic Team identifier: <code>{eventId}-T{XX}</code> .
<code>teamName</code>	String	No	Team display name (e.g., "Project Nexus").
<code>leader</code>	String	No	Full name of the primary team leader.
<code>email</code>	String	Yes	Leader contact email address.
<code>createdAt</code>	Number	No	Millisecond timestamp when record was committed.
<code>attendanceMarked</code>	Boolean	No	Legacy Day 1 attendance flag maintained for backward compatibility.
<code>position</code>	Number	Yes	Podium placement: 1 (1st Place), 2 (2nd Place), 3 (3rd Place), or null.
<code>fcmToken</code>	String	Yes	Firebase Cloud Messaging device registration token for Web Push.
<code>fcmTokenUpdatedAt</code>	Number	Yes	Timestamp when the device token was updated.
<code>members</code>	Array	No	Array of <code>TeamMember</code> objects (<code>name</code> , <code>rollNumber</code> , <code>college</code> , <code>branch</code> , <code>present</code>).
<code>qualifications</code>	Map	Yes	Key-value pairs mapping round string to boolean (<code>{"1": true, "2": false}</code>).
<code>dayAttendance</code>	Map	Yes	Key-value pairs mapping day index to <code>DayAttendance</code> records.

7.4 ENTITY-RELATIONSHIP (ER) / TREE MODEL



7.5 KEY NORMALIZATION, KEYS, AND SANITIZATION RULES

Firebase Realtime Database node keys possess strict character restrictions. A key string cannot contain `.`, `#`, `$`, `[`, `]`, `:`, or `/`. Violating these rules throws runtime exceptions. Eventra implements strict sanitization algorithms:

7.5.1 Email Key Sanitization (`formatEmailForDb`)

Emails contain periods (`.`). The helper function normalizes email keys by substituting periods with commas:

```
export const formatEmailForDb = (email: string): string => {
  return email.toLowerCase().replace(/\./g, ',');
};
```

Example: `alice.developer@college.edu` $\rightarrow$ `alice,developer@college,edu`.

7.5.2 FCM Token Sanitization (`sanitizeTokenKey` / `sanitizeKey`)

Cryptographic push registration tokens contain colons, slashes, and periods. The sanitization utility transforms all illegal characters to underscores:

```
export function sanitizeTokenKey(token: string): string {
  return token.replace(/[\.$\/:]/g, '_');
}
```

7.6 ATOMIC TRANSACTIONS AND CONCURRENCY HANDLING

In standard database systems, updating a team count involves a read-modify-write cycle: $\text{Read } C \rightarrow \text{Compute } C + 1 \rightarrow \text{Write } C + 1$. If two participants register simultaneously at capacity $C = 99$ with maximum limit 100 , both clients read 99 , compute 100 , and write 100 . As a result, 101 teams are registered, violating the institutional quota constraint.

Eventra resolves this race condition using Firebase atomic transactions (`runTransaction`):

```
const result = await runTransaction(ref(db, `events/${eventId}/eventSettings/currentTeams`),
  (current) => {
    const count = current || 0;
    if (eventSettings?.maxTeams && count >= eventSettings.maxTeams) {
      return; // Aborts transaction atomically
    }
    return count + 1; // Commits increment atomically
  });

if (!result.committed) {
  setErrors({ submit: 'Registration full. No more teams can be accepted.' });
  return;
}
```

If another transaction commits concurrently, the Firebase engine automatically restarts the transaction with the updated value, guaranteeing mathematical consistency.

7.7 CRUD OPERATIONS ANALYSIS

- **Create:**

- Provision event: `set(ref(db, 'events/' + eventId), { passwordHash, createdAt, teamCount: 0 })`.
- Register team: Atomic counter increment followed by multi-path batch `set()`.
- Queue broadcast: `push(ref(db, 'notificationQueue/' + eventId), payload)`.
- **Read:**
 - Real-time listener: `onValue(ref(db, path), callback)`.
 - One-time read: `get(ref(db, path))` wrapped in `withRetry()`.
- **Update:**
 - Update settings: `update(ref(db, 'events/' + eventId + '/eventSettings'), changes)`.
 - Mark attendance: `update(ref(db, 'events/' + eventId + '/teams/' + code), { 'dayAttendance/1/checked': true })`.
 - Qualify round: `update(ref(db, 'events/' + eventId + '/teams/' + code), { 'qualifications/2': true })`.
- **Delete:**
 - Revoke FCM token: `db.ref('fcmTokens/teams/' + eventId + '/' + code).remove()`.
 - Clean dedup record: `db.ref('notificationLog/' + key).remove()`.

7.8 DATA VALIDATION RULES AND INTEGRITY CONSTRAINTS

- **Event ID:** Enforced via regex: `/^[a-z0-9]+(-[a-z0-9]+)*$/` with minimum 3 and maximum 40 characters (`isValidEventId`).
- **Team Size Bounds:** Dynamically constrained such that: `$$1 \le \text{teamSizeMin} \le \text{teamSizeMax} \le 10 $$ $ $\text{teamSizeMin} \le (\text{members.length} + 1) \le \text{teamSizeMax} $$`
- **Email Uniqueness:** Enforced by writing to an index table `events/{eventId}/registeredEmails/{emailKey}`.

7.9 DATABASE SECURITY AND ACCESS RULES

Database security is structured around event scoping. In development and managed production setups, security rules enforce:

- Administrative nodes require matching session contexts.
 - Write access to `registeredEmails` and `teams` allows creation only when non-existent (preventing overwrite of existing teams).
 - Server microservice uses administrative Service Account credentials, bypassing security rule bottlenecks on background tasks.
-

CHAPTER 8 — BACKEND DEVELOPMENT

8.1 BACKEND OVERVIEW

The backend tier of Eventra is implemented as an independent, autonomous microservice located in the `notification-server/` directory. It is purpose-built to execute background tasks that cannot reliably run on client browsers, such as scheduled deadline monitoring, stale push token cleanup, and automated notification queue dispatch.

Unlike typical monolithic backends that manage full database CRUD, Eventra's notification server is architected as an **Event-Driven Scheduled Background Worker**. It operates alongside the client SPA and leverages the privileged **Firebase Admin SDK** to interface with the Realtime Database and Google's Firebase Cloud Messaging gateway.

8.2 SERVER INITIALIZATION AND EXPRESS CONFIGURATION

The main entry point is `notification-server/index.js`. It initializes an Express server instance on the port defined by `process.env.PORT` (defaulting to 3001).

Key configuration features include:

- **CORS Middleware:** Configured to permit requests from any origin with support for `GET`, `POST`, and `OPTIONS` preflight checks.
- **JSON Body Parser:** Configured via `express.json()` to parse incoming notification dispatch payloads.
- **Boot Diagnostics:** Measures server start timestamp (`startTime`) to report uptime metrics on health endpoints.
- **Cron Initialization:** Triggers `startCronJobs()` immediately after the HTTP listener binds successfully.

8.3 MIDDLEWARE IMPLEMENTATION

- **CORS Handling:**

```
app.use(cors({
  origin: '*',
  methods: ['GET', 'POST', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
app.options('*', cors());
```

- **Job Execution Wrapper (`wrapJob`):** Every cron job function is wrapped in an execution supervisor that tracks total run counts, logs timestamps, and traps unhandled exceptions without crashing the Express server:

```
function wrapJob(name, jobFn) {
  return async () => {
    try {
      cronStatus[name].runs++;
      cronStatus[name].lastRun = new Date().toISOString();
      await jobFn();
    } catch (err) {
      cronStatus[name].errors++;
      console.error(`[Cron] Job "${name}" failed:`, err.message);
    }
  };
}
```

8.4 FIREBASE ADMIN SDK SERVER-SIDE INTEGRATION

Server-side authentication with Firebase is managed by `notification-server/lib/firebase.js`. To guarantee deployment flexibility across local development machines and cloud PaaS environments (like Render), the service account credentials locator implements a robust four-tier resolution hierarchy (`getServiceAccount()`):

1. **Local File Detection:** Checks for candidate filenames (`serviceAccountKey.json`, `service-account.json`) in the server directory or workspace root.
2. **Path Environment Variable:** Checks `FIREBASE_SERVICE_ACCOUNT_PATH` or `GOOGLE_APPLICATION_CREDENTIALS`.
3. **Raw String / Base64 Variable:** Inspects `FIREBASE_SERVICE_ACCOUNT`, dynamically detecting whether the content is raw JSON or Base64-encoded JSON.
4. **Individual Environment Variables:** Synthesizes credentials from `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL`, and `FIREBASE_PRIVATE_KEY`.

Upon credential parsing, the Admin SDK initializes:

```
admin.initializeApp({
  databaseURL: process.env.FIREBASE_DATABASE_URL,
  credential: admin.credential.cert(serviceAccount),
});
```

8.5 NOTIFICATION DISPATCH ENGINE

The core notification dispatch logic resides in `notification-server/lib/notifications.js`. It exposes high-level targeting functions that construct Web Push payloads and deliver them through the Firebase Cloud Messaging API (`admin.messaging().send(message)`).

8.5.1 Single Token Dispatch (`sendToToken`)

Constructs a standards-compliant FCM message containing both standard notification fields and `webpush` options (icon, badge, link, data).

8.5.2 Stale Token Eviction Algorithm

When an FCM message send attempt fails, the engine analyzes the resulting error code. If the code indicates an invalid or expired token (`messaging/registration-token-not-registered`, `messaging/invalid-registration-token`):

1. The token is marked invalid.
2. `cleanTeamToken(eventId, teamCode)` is invoked.
3. The invalid token is scrubbed from both `fcmTokens/teams/{eventId}/{teamCode}` and `events/{eventId}/teams/{teamCode}/fcmToken`.
4. The internal `totalCleaned` counter increments.

8.5.3 Target Filtering Functions

- `sendToEventTeams(eventId, payload)`: Iterates over all registered teams under `fcmTokens/teams/{eventId}/`.
- `sendToSpecificTeams(eventId, teamCodes, payload)`: Sends notifications strictly to an array of specified team codes.
- `sendToQualifiedTeams(eventId, round, payload)`: Inspects `events/{eventId}/teams`, filters teams where `qualifications[round] === true`, and dispatches push alerts.
- `sendToWinnerTeams(eventId, payload)`: Filters teams where `position > 0` (1st, 2nd, 3rd place) and delivers victory notifications.

8.6 DEDUPLICATION LOGGING SUBSYSTEM

Automated background cron jobs execute at high frequencies (every 1 to 2 minutes). Without synchronization, a cron job evaluating a deadline condition would fire redundant push notifications every minute.

The deduplication subsystem in `notification-server/lib/dedup.js` resolves this:

- `hasBeenSent(key)`: Checks if an entry exists at `notificationLog/{sanitizedKey}`.
- `markAsSent(key, metadata)`: Writes an audit record containing `sentAt`, `key`, and metadata.
- **Dynamic Key Generation**: When an organizer changes an event deadline, the deadline ISO string is embedded directly in the deduplication key: `$$\text{dedupKey} = \text{eventId} \parallel \text{"_reg_closing_24h_"} \parallel \text{registrationDeadline}$$`. If the deadline is modified, the key hash changes automatically, allowing fresh reminders to fire for the new deadline schedule.
- `clearForEvent(eventId, type)`: Programmatically purges deduplication records when deadlines are extended.
- `cleanupOldRecords(daysOld)`: Purges audit logs older than 7 days daily at 3:00 AM.

8.7 SCHEDULED CRON JOBS ARCHITECTURE

The server registers five recurring tasks using `node-cron`:



8.7.1 Job 1: Registration Reminders (`jobs/registration-reminders.js`)

- **Frequency:** Every minute (`* * * * *`).
- **Logic:** Iterates over all events where `registrationOpen === true` and a valid `registrationDeadline` exists.
- **Window Tolerance:** Evaluates if current time is within ± 2 minutes of either the **24-hour mark** ($\$86,400,000 \text{ ms}$) or the **1-hour mark** ($\$3,600,000 \text{ ms}$) before deadline passage.
- **Dispatch:** Sends push reminders to all registered teams with direct links to the registration page.

8.7.2 Job 2: Registration Status Monitor (`jobs/registration-status.js`)

- **Frequency:** Every minute (`* * * * *`).
- **Logic:** Maintains an in-memory state cache (`stateCache`) per event tracking `registrationOpen`, `registrationDeadline`, and `deadlinePassed`.
- **Transitions Detected:**
 - *Status Toggle:* Detects when an organizer manually closes registration, broadcasting `Registration Closed`.
 - *Deadline Modification:* Detects when an organizer extends a deadline, clears existing reminder dedup records, and broadcasts `Schedule Update: Deadline Extended`.
 - *Deadline Passage:* Detects the exact minute a deadline passes, broadcasting `Registration Closed`.

8.7.3 Job 3: Qualification Notifications (`jobs/qualification-notifications.js`)

- **Frequency:** Every 2 minutes (`*/2 * * * *`).
- **Logic:** Maintains in-memory caches of team qualifications (`qualCache`) and winner positions (`winnerCache`).
- **Triggers:**
 - *Winner Placement:* Detects when a team is assigned position 1, 2, or 3, broadcasting personalized victory notifications (e.g., "🏆 Congratulations on 1st Place!").
 - *Stage Advancement:* Detects when a team's qualification status flips from `false` to `true` for any round, dispatching celebratory advancement notifications.

8.7.4 Job 4: Organizer-Queued Push Worker (`jobs/organizer-push.js`)

- **Frequency:** Every 30 seconds (`*/30 * * * * *`).
- **Logic:** Inspects `notificationQueue/{eventId}/`.
- **Execution:** Drains unprocessed queue items, routes payloads to the appropriate targeting handler (`all_teams`, `qualified_round`, `winners`, `specific_teams`), marks items as `processed: true`, and logs delivery success/failure counts.

8.8 MANUAL BROADCAST & WEBHOOK ENDPOINTS

In addition to background cron workers, the server exposes HTTP endpoints:

- `POST /notify`: Accepts a JSON payload ({ `eventId`, `title`, `body`, `target`, `targetRound`, `teamCodes`, `url` }) to trigger an immediate manual push broadcast outside the cron loop.

8.9 COMPLETE BACKEND API ENDPOINTS TABLE

Method	Route	Authentication Required	Request Body / Params	Expected Response	Error Responses
GET	<code>/</code>	None	None	Service info, version, uptime, cron run statistics	500 on internal failure
GET	<code>/status</code>	None	None	Notification counts (<code>totalSent</code> , <code>totalFailed</code> , <code>totalCleaned</code>), cron status map	500 on internal failure
GET	<code>/debug</code>	None	None	Service account status, database URL, RTDB connection health (<code>.info/connected</code>)	500 on internal failure
POST	<code>/notify</code>	Secret / Origin	JSON: <code>eventId</code> , <code>title</code> , <code>body</code> , <code>target</code> , <code>targetRound</code> , <code>teamCodes</code> , <code>url</code>	JSON: { <code>success</code> : true, <code>sent</code> : N, <code>failed</code> : M }	400 (Missing fields), 500 (Dispatch failure)

8.10 ERROR HANDLING AND PROCESS RECOVERY

- **Database Disconnection Handling:** Database operations use asynchronous promises with catch blocks. If RTDB connectivity drops, cron runs log a warning and retry on the subsequent scheduled cycle.
- **Uncaught Exception Resilience:** Process-level unhandled rejections are caught in `wrapJob`, preventing Node process termination.

CHAPTER 9 — FRONTEND DEVELOPMENT

9.1 FRONTEND OVERVIEW AND UI DESIGN SYSTEM

The frontend of Eventra is constructed as a modern, high-speed Single Page Application (SPA). The design philosophy balances the visual elegance of luxury typography with the technical precision required for intense hackathon environments.

Key aesthetic characteristics include:

- **Vintage Tech & Ambient Luxury Dark Theme:** Dark slate canvas (`#080810` , `#0a0a0f`) layered with gold accents (`#C6A969` , `#D4AF37`) and subtle green presence indicators (`#4ADE80`).
- **Glassmorphism & Depth:** Translucent content cards featuring `backdrop-filter: blur(16px)` and delicate `$1\text{px}` gold-tinted borders (`rgba(198,169,105,0.15)`).
- **Curated Font Pairing:** `Crimson Pro` serif for distinguished headings paired with `JetBrains Mono` for data tables, codes, and metrics.
- **Skeleton Shimmer Loading:** Instead of abrupt loading spinners, content-matching animated skeletons (`Skeleton.tsx`) preserve spatial layout while data streams in from the cloud.

9.2 APPLICATION DIRECTORY STRUCTURE

The frontend codebase is organized cleanly within the `src/` directory:

```

src/
├── assets/                # Static graphic assets (hero.png, logos)
├── components/            # Reusable UI widgets & domain components
│   ├── Button.tsx        # Standard button with variants (primary, secondary, danger, ghost)
│   ├── CSVDownloadModal.tsx # Multi-option analytical CSV export modal
│   ├── GlassCard.tsx     # Translucent glassmorphism container
│   ├── Input.tsx         # Form input with validation error states
│   ├── Layout.tsx        # Base page wrapper with ambient lighting & footer
│   ├── LoadingSpinner.tsx # Custom circular loading spinner
│   ├── Navbar.tsx        # Fixed navigation header with route highlights
│   ├── NotificationPanel.tsx # Organizer push broadcast compose console
│   ├── NotificationPrompt.tsx # Foreground in-app push notification toast listener
│   ├── ProtectedRoute.tsx # Route authentication guard
│   ├── QRCodeDisplay.tsx  # Canvas 2D QR rendering and download widget
│   ├── QRScanner.tsx     # Camera optical barcode scanning engine
│   ├── Skeleton.tsx      # Layout-matching shimmer loading placeholders
│   ├── StatusBadge.tsx   # Presence badges (Present / Absent / Pending)
│   ├── SuccessAnimation.tsx # Checkmark scale-in animation
│   ├── Textarea.tsx      # Multi-line text entry field
│   └── TicketCard.tsx    # Printable luxury boarding pass component
├── context/
│   └── AuthContext.tsx   # Organizer session context backed by sessionStorage
├── lib/
│   ├── constants.ts      # Global app constants & APPROVAL_KEY binding
│   ├── db-retry.ts       # Fault-tolerant exponential backoff retry helper
│   ├── fcm.ts            # Client-side Firebase Cloud Messaging & token logic
│   ├── firebase.ts       # Client Firebase App & RTDB initialization
│   ├── haptics.ts        # Hardware vibration feedback patterns
│   └── utils.ts          # Cryptographic SHA-256, formatting, CSV builders
├── pages/
│   ├── CreateEvent.tsx   # Provision new event with approval key
│   ├── Dashboard.tsx     # Central organizer command dashboard
│   ├── EventDetails.tsx  # Configure days, rounds, dates, deadlines
│   ├── Home.tsx          # Interactive public landing page & features showcase
│   ├── Leaderboard.tsx   # Public tournament leaderboard & GTA V overlay
│   ├── OrganizerLogin.tsx # Event authentication login portal
│   ├── Register.tsx      # Participant team registration portal
│   ├── RegistrationSuccess.tsx # Confirmation screen with boarding pass
│   ├── ScanAttendance.tsx # Multi-day optical QR check-in camera tool
│   └── Ticket.tsx        # Standalone public boarding pass view
├── styles/
│   └── eventra-shared.css # Keyframe animations, utility pills, custom inputs
├── types/
│   └── index.ts          # Complete TypeScript data model interfaces
├── App.css               # Component-specific stylesheet
├── App.tsx               # Client router definition and route guards
├── index.css             # Base Tailwind theme, color variables, utility classes
└── main.tsx              # React application root mounting

```

9.3 CLIENT-SIDE ROUTING AND NAVIGATION

Client routing is powered by **React Router DOM v7**. Routes are structured into public views and protected organizer views:

```

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route element={<Layout />>
      { /* Public Routes */}
      <Route path="create-event" element={<CreateEvent />} />
      <Route path="organizer-login" element={<OrganizerLogin />} />
      <Route path="register/:eventId" element={<Register />} />
      <Route path="registration-success/:eventId/:teamId" element={<RegistrationSuccess />} />
      <Route path="ticket/:eventId/:teamId" element={<Ticket />} />
      <Route path="leaderboard/:eventId" element={<Leaderboard />} />

      { /* Protected Organizer Routes */}
      <Route path="event-details/:eventId" element={<ProtectedRoute><EventDetails />
</ProtectedRoute>} />
      <Route path="dashboard/:eventId" element={<ProtectedRoute><Dashboard /></ProtectedRoute>}
/>
      <Route path="scan/:eventId" element={<ProtectedRoute><ScanAttendance /></ProtectedRoute>}
/>

      <Route path="*" element={<Navigate to="/" replace />} />
    </Route>
  </Routes>
</BrowserRouter>

```

9.4 SESSION MANAGEMENT & AUTHCONTEXT

Administrative authentication state is managed via React Context (`src/context/AuthContext.tsx`):

- **Storage Mechanism:** Backed by `window.sessionStorage` under the key `eventra_organizer_session`.
- **Persistence Boundary:** The session survives page refreshes but is automatically destroyed when the browser tab is closed.
- **Event Scoping:** Stores `{ isAuthenticated: true, eventId }`.
- **Route Guard Verification (`ProtectedRoute.tsx`):** Inspects whether `isAuthenticated === true` and verifies that the route's `:eventId` parameter matches the active session. If an organizer logged into event `event-A` attempts to access `/dashboard/event-B`, an "Access Denied" view is displayed with a switch button.

9.5 PAGE MODULES IMPLEMENTATION

9.5.1 Landing Page (`Home.tsx`)

A high-impact public introduction to Eventra featuring dynamic navbar scroll transitions, an animated ambient glow hero, live platform statistic counters, feature grid cards with micro-interactions, an interactive step-by-step workflow preview, simulated boarding pass ticket widgets, and benefits breakdown.

9.5.2 Event Creation (`CreateEvent.tsx`)

A multi-field form validating the secret master approval key (`APPROVAL_KEY`), event ID regex, and password confirmation. Upon submission, it queries RTDB to guarantee the `eventId` is unique, hashes the password client-side using SHA-256, and provisions the initial database node.

9.5.3 Organizer Login (`OrganizerLogin.tsx`)

Authenticates organizers by retrieving the stored password hash from `events/{cleanId}/passwordHash`, hashing the input password client-side, and verifying equality before invoking `login(cleanId)`.

9.5.4 Event Configuration (`EventDetails.tsx`)

Administrative portal for editing event titles, descriptions, venue, schedules, and registration deadlines. Notably incorporates visual stepper buttons to adjust:

- `numberOfDays` and `currentDay` (Attendance subsystem).
- `numberOfRounds` and `currentRound` (Qualification subsystem).

9.5.5 Participant Registration (`Register.tsx`)

The public registration form. Evaluates whether registrations are open or deadlines have passed. Dynamically renders member input blocks with "Same college as leader" and "Same branch as leader" replication shortcuts. Upon submission, executes atomic counter locking, writes team records, and registers FCM push tokens.

9.5.6 Registration Confirmation (`RegistrationSuccess.tsx`)

Post-registration confirmation screen displaying animated celebration rings, instructions for arrival, and embedding the luxury `TicketCard` boarding pass.

9.5.7 Organizer Dashboard (`Dashboard.tsx`)

The central operations console containing four dedicated views:

1. **Teams Tab:** Searchable list of all registered teams with per-day attendance dots, round qualification tags, expandable member rosters, and copyable FCM debug tokens.
2. **Qualified Tab:** Interactive round-by-round management tool. Allows organizers to toggle qualification stars for non-final rounds, or award 1st, 2nd, and 3rd place podium medals in the final round.
3. **Lookup Tab:** Direct alphanumeric Team ID search returning the team's QR code and member status.
4. **Push Broadcast Tab:** Houses the `NotificationPanel` for composing targeted push notifications.

9.5.8 Attendance Scanner (`ScanAttendance.tsx`)

Mobile-optimized gate check-in console displaying the active event day badge. Renders the camera QR scanner, decodes tickets, parses the `{eventId}|{teamId}` payload, verifies against RTDB, guards against duplicates, renders individual member checkboxes, and commits attendance with millisecond timestamps.

9.5.9 Tournament Leaderboard (`Leaderboard.tsx`)

Real-time public rankings display. Features round progress pills, search filtering, finalist counts, podium reveal interactions with confetti bursts, and the GTA V "WASTED" elimination dramatization.

9.6 SPECIALIZED UI COMPONENTS

9.6.1 Luxury Boarding Pass Virtualizer (`TicketCard.tsx`)

An airline-style boarding pass styled with luxury gold accents, flight cutouts, barcode notches, and destination labels. It embeds `QRCodeDisplay` and utilizes `html2canvas` to capture the DOM tree and trigger an instant PNG download named `{eventId}-boarding-pass-{teamId}.png`. Explicit color fills are applied to prevent gradient text-fill leakage during canvas rasterization.

9.6.2 Optical QR Scanner (`QRScanner.tsx`)

Encapsulates `Html5QrcodeScanner` within a React lifecycle ref. Configured for 10 FPS scanning, a 250×250 px bounding box, native camera switching, and torch toggle support. Cleans up video streams cleanly upon component unmount.

9.6.3 Analytical CSV Export Modal (`CSVDownloadModal.tsx`)

An accessible modal overlay providing one-click exports of:

- All Details CSV (Full team data, roll numbers, branches, per-round qualifications, per-day attendance).
- Round-Qualified CSVs (One dedicated dataset per round).
- Day-Attendance CSVs (One dedicated dataset per event day).

9.7 FORM MANAGEMENT, VALIDATION, AND DYNAMIC FIELDS

Form handling utilizes controlled React state with field-level validation:

- Input changes trigger instant error clearance.
- Email fields validate standard RFC 5322 regex formatting.
- Dynamic member arrays support real-time addition/removal while respecting `teamSizeMin` and `teamSizeMax`.

9.8 HARDWARE INTEGRATION: VIBRATION HAPTICS & AUDIO

Eventra bridges web software with mobile physical hardware:

- **Haptic Feedback (`src/lib/haptics.ts`):** Wraps `window.navigator.vibrate()` into expressive tactile profiles:
 - `light` : 10 ms pulse for checkbox clicks and toggles.
 - `medium` : 20 ms pulse for standard button taps.
 - `heavy` : 45 ms pulse for significant actions.
 - `success` : Rhythmic pulse `[10, 50, 15]` when attendance is confirmed.

- `error` : Triple jolt `[50, 100, 50]` on duplicate or invalid scans.
 - `celebration` : Multi-pulse burst `[60, 40, 60, 40, 100]` matching confetti firing.
 - `wasted` : Fading throb sequence `[120, 60, 80, 80, 60, 100, 40, 120, 30]` matching the GTA screen shake.
- **Audio Feedback:** The leaderboard initializes an audio element loading `/gta-v-death-sound-effect-102.mp3` when an eliminated team is searched, playing the iconic death sound.

9.9 SERVICE WORKER & FOREGROUND NOTIFICATION RECEPTION

Push message reception operates in two distinct operational states:

1. **Background / Tab Closed:** Managed by `public/firebase-messaging-sw.js`. The Service Worker receives raw FCM payloads via `messaging.onBackgroundMessage()` and triggers native OS system tray notifications via `self.registration.showNotification()`.
 2. **Foreground / Tab Active:** Managed by `NotificationPrompt.tsx` using `onMessage(messaging, callback)`. Instead of interrupting the user with OS popups, it displays a luxury floating toast in the top-right corner, accompanied by a light haptic pulse, automatically dismissing after 7 seconds.
-